

Tóm tắt điều hành

Báo cáo này phân tích và hoàn thiện thiết kế cơ sở dữ liệu (CSDL) cho ứng dụng quản lý trọ dựa trên sơ đồ hiện tại và các yêu cầu chức năng đã nêu. Phân tích cho thấy CSDL hiện tại còn thiếu nhiều thực thể cần thiết (như **bảng Khách thuê (Tenants), hóa đơn chi tiết (InvoiceItems), giao dịch thanh toán (Payments), công nợ (Debts), thông báo (Notifications), audit log...**), cũng như các trường bắt buộc (ví dụ, `tenant_id` để hỗ trợ đa cơ sở, `created_at/created_by` cho audit, v.v.). Chúng tôi đề xuất thêm các bảng và quan hệ mới, chuẩn hóa mô hình (theo **chuẩn NF**), và bổ sung các ràng buộc khóa để đảm bảo toàn vẹn. Mỗi bảng được gán với các chỉ mục quan trọng (ví dụ, chỉ mục theo `tenant_id`, `user_id`, ngày tháng báo cáo) để tối ưu truy vấn báo cáo ¹ ². Hệ thống cũng sử dụng tính năng **Row-Level Security** của PostgreSQL để phân tách dữ liệu theo khách thuê (nếu cần hỗ trợ đa-tenant) ³ ².

Các đoạn code mẫu DDL bên dưới tạo bảng cơ bản (Users, Roles, Buildings, Rooms, Tenants, Contracts, Invoices, InvoiceItems, UtilityReadings, Services, Vehicles, TenantVehicles, Payments, Debts, Notifications, AuditLogs, RoomPhotos) với khóa chính/phụ phù hợp, các enum trạng thái, và các trường *audit* (`created_at`, `updated_at`, `deleted_at`, `created_by`, `updated_by` theo khuyến nghị ⁴ ¹). Quy trình tính tiền tự động được minh họa bằng ví dụ cụ thể 3 phòng, tính toán tiền thuê, tiền điện-nước và dịch vụ kèm VAT/cước, làm tròn, và phân kỳ (nếu khách vào giữa tháng). Khi tạo hóa đơn, hệ thống sẽ sinh **QR code** mã hóa số tiền; khi thanh toán, cổng thanh toán (VD: ZaloPay) sẽ **gọi callback** thông báo kết quả ⁵, sau đó phần mềm đối soát và gạch nợ tự động. Tính năng nhắc thanh toán có thể tận dụng Zalo OA (ZNS) để gửi template tới khách ⁶. Cuối cùng, đề xuất các endpoint RESTful cơ bản cho từng chức năng, chiến lược nhập liệu từ Excel (xử lý CSV, kiểm tra hợp lệ, idempotency), và quy trình di cư dữ liệu từ hệ thống cũ (migration) đã được soạn thảo chi tiết.

Phân tích hiện tại và yêu cầu chức năng (Bảng so sánh)

Mô hình hiện tại có các bảng: `DEVICE`, `ROOM`, `USER`, `ROLE`, `UtilityUsage`, `Vehicle`, `Service`, `Building`, `Contract`, `Invoice`. So với các tính năng, chúng tôi nhận thấy nhiều điểm thiếu/không hợp lý như sau:

- **Quản lý phòng (Rooms):** Bảng `ROOM` hiện tại có `room_id`, `building_id`, `price`, `status`, ... nhưng nên bổ sung thêm trường trạng thái chi tiết (ví dụ `available`, `occupied`, `maintenance`). Nên có ràng buộc khóa ngoại `building_id` tham chiếu `BUILDING.id` ¹.
- **Quản lý khách thuê (Tenants):** Hiện CSDL chỉ có bảng `USER` (có thể dùng cho chủ trọ/quản trị). Tuy nhiên cần bảng riêng `TENANTS` để lưu thông tin khách (Họ tên, CCCD, số ĐT, ngày vào ở, tiền cọc, ảnh CCCD, ghi chú, v.v.) và liên kết với `ROOM` hoặc `CONTRACT`. Thiếu bảng này là điểm quan trọng nhất.
- **Quản lý hợp đồng (Contracts):** Bảng `CONTRACT` có thể thiếu các thông tin như `start_date`, `end_date`, `deposit_amount`, `status` (đang hiệu lực/đã hết hạn/đã thanh lý). Cần liên kết `contract.tenant_id -> TENANTS.id`, `contract.room_id -> ROOM.id`. Hệ thống nên nhắc khi hợp đồng gần hết hạn.
- **Hóa đơn và chi tiết hóa đơn (Invoices, InvoiceItems):** Bảng `INVOICE` chỉ lưu tổng tiền; cần thêm bảng `INVOICE_ITEMS` để lưu chi tiết các mục cước (tiền phòng, tiền điện, nước, wifi, dịch vụ khác), mỗi mục có `description`, `amount`, `VAT%` nếu có. Quan hệ 1:N giữa `INVOICES`

và `INVOICE_ITEMS`. Thiết kế này giúp audit theo Redgate ⁷ (mỗi hóa đơn lưu snapshot chi tiết thời điểm lập).

- **Chỉ số điện nước (UtilityReadings):** Bảng `UtilityUsage` hiện có thể lưu chỉ số. Cần ràng buộc `utility_readings.room_id -> ROOM.id`, `reading_date`, `previous_reading`, `current_reading`. Quy trình: Nhập chỉ số đầu/cuối kỳ, phần mềm tự tính `usage*unit_price`. Có thể đưa `unit_price` vào bảng `BUILDING` hoặc `ROOM`.
- **Dịch vụ kèm theo (Services):** Bảng `Service` liệt kê loại dịch vụ (wifi, vệ sinh, gửi xe, v.v.) và giá. Cần bảng phụ `ContractServices` hoặc tính trực tiếp trong hóa đơn (qua `INVOICE_ITEMS`). Ví dụ, mỗi `CONTRACT` có thể chọn các dịch vụ cố định, khi lập hóa đơn sẽ tự cộng phí theo mức quy định.
- **Quản lý xe (Vehicles):** Bảng `Vehicle` hiện chỉ lưu thông tin xe. Để liên kết với khách thuê, đề xuất thêm bảng `TENANTS_VEHICLES` với các trường (`tenant_id`, `vehicle_id`, `license_plate`, `fee`, `photo`, ...). Quan hệ N:1 giữa `TENANTS` và `VEHICLE` (mỗi khách có thể có nhiều xe).
- **Thanh toán và công nợ (Payments, Debts):** CSDL hiện chưa có bảng ghi nhận giao dịch thanh toán. Cần bảng `PAYMENTS` (ghi `payment_id`, `invoice_id`, `amount`, `method`, `paid_at`, `reference`). Đồng thời, bảng `DEBTS` hoặc cờ nợ trên `INVOICES` để lưu số tiền còn thiếu (nợ) sau hạn thanh toán. Cần tự động tổng hợp nợ của từng phòng ⁸ ⁶.
- **Nhắc thanh toán (Notifications):** Cần bảng `NOTIFICATIONS` để ghi các thông báo gửi qua Zalo/App (bao gồm `tenant_id`, `type`, `message`, `sent_at`). Hệ thống lập hóa đơn xong có thể đẩy hàng loạt thông báo nhắc thanh toán (ví dụ dùng API Zalo OA – ZNS) ⁶.
- **Phân quyền và người dùng (Users, Roles):** Bảng `USER` / `ROLE` cần bổ sung cơ chế phân quyền đa tenant. Có thể giữ `roles` toàn cục hoặc tốt hơn là thêm `tenant_id` trong `roles` để các vai trò gắn với từng chủ trọ riêng (theo mô hình WorkOS ⁹ ¹⁰). Đồng thời thêm bảng `user_roles(user_id, tenant_id, role_id)` để gán vai trò cho người dùng trong từng tenant.
- **Audit/log:** Mỗi bảng quan trọng nên có các trường `created_at` `TIMESTAMP`, `updated_at` `TIMESTAMP`, `created_by`, `updated_by`, `deleted_at` theo khuyến cáo ⁴ ¹¹ để ghi nhận lịch sử thao tác và thực hiện *soft-delete*.
- **Ảnh phòng (RoomPhotos):** Không có bảng lưu ảnh. Đề xuất thêm `ROOM_PHOTOS(id, room_id, photo_url, description)` nếu cần lưu hình phòng (useful cho đăng tin tìm khách).

Dưới đây là bảng so sánh tóm tắt giữa mô hình hiện tại và đề xuất cải tiến (chỉ nêu những thành phần chính):

Yêu cầu/Tính năng	Thiết kế hiện tại	Đề xuất bổ sung/Điều chỉnh
Khách thuê (Tenants)	Không có (dùng chung với <code>USER</code> ?)	Tạo bảng <code>TENANTS(id, name, cccd, phone, deposit, checkin_date, ...)</code> , liên kết <code>ROOM</code> , <code>CONTRACT</code>
Phòng (Rooms)	Bảng <code>ROOM(room_id, building_id, price, status, ...)</code>	Thêm trường trạng thái chi tiết (<code>available/occupied/maintenance</code>), giá Internet, phí dịch vụ cố định, chỉ mục <code>status</code> tìm kiếm nhanh ¹

Yêu cầu/Tính năng	Thiết kế hiện tại	Đề xuất bổ sung/Điều chỉnh
Hợp đồng (Contracts)	<code>CONTRACT(contract_id, room_id, user_id, ...)</code> (chưa rõ các trường)	Thêm <code>tenant_id</code> , <code>start_date</code> , <code>end_date</code> , <code>deposit_amount</code> , <code>status</code> , ràng buộc FK <code>tenant_id -> TENANTS(id)</code>
Hóa đơn (Invoices)	<code>INVOICE(invoice_id, contract_id, total_amount, ...)</code>	Thêm <code>invoice_date</code> , <code>due_date</code> , <code>status_paid</code> , liên kết <code>contract</code> . Phân chia thành <code>INVOICE_ITEMS</code> chi tiết (tiền phòng, điện, nước, dịch vụ) ⁷
Chỉ số điện nước	<code>UtilityUsage(room_id, date, usage)</code>	Ràng buộc <code>room_id</code> , lưu <code>prev_reading</code> , <code>cur_reading</code> , tự tính <code>usage*(đơn giá)</code> theo tháng
Dịch vụ phụ trội (Services)	<code>SERVICE(service_id, name, price, type, ...)</code>	Cần bảng này. Có thể thêm bảng <code>CONTRACT_SERVICES(contract_id, service_id)</code> nếu một hợp đồng chọn dịch vụ. Lập hóa đơn cộng phí tự động theo mức sử dụng/định mức.
Xe của khách (Vehicles)	<code>Vehicle(vehicle_id, type, license, color, ...)</code>	Thêm FK <code>tenant_id -> TENANTS(id)</code> , hoặc tách thành <code>TENANTS_VEHICLES(tenant_id, vehicle_id, plate, fee, photo, ...)</code> để liên kết nhiều xe với khách.
Thanh toán/ Đối soát (Payments)	Không có bảng lưu giao dịch	Thêm <code>PAYMENTS(payment_id, invoice_id, amount, method, paid_at, reference)</code> . Số tiền khách thanh qua QR sẽ so khớp vào đây tự động (xem bên dưới).
Công nợ (Debts)	Không có	Thêm <code>DEBTS(id, invoice_id, amount_due, due_date, status)</code> hoặc ánh xạ tồn nợ từ <code>INVOICES</code> . Hệ thống tổng hợp nợ từng phòng/khách ⁶ ⁸ .
Thông báo (Notifications)	Không có	Thêm <code>NOTIFICATIONS(id, tenant_id, type, content, scheduled_time, sent_at)</code> . Dùng Zalo OA (ZNS) để gửi nhắc tiền tự động.
Audit logs	Không có	Thêm <code>AUDIT_LOG(id, table_name, record_id, action, old_values, new_values, timestamp, user_id)</code> . Mỗi bảng có thêm các cột audit (<code>created_at/by</code> , v.v.) ⁴ ¹ .

Yêu cầu/Tính năng	Thiết kế hiện tại	Đề xuất bổ sung/Điều chỉnh
Phân quyền (Roles)	ROLE(role_id, name) + USER (user_id, role_id ?)	Cần bổ sung ràng buộc tenant: ví dụ ROLE(id, tenant_id, name) và bảng USER_ROLES(user_id, tenant_id, role_id) ⁹ ¹² . Tất cả truy vấn phân quyền gắn kèm tenant_id.
Phòng trống/Đăng tin	Không có trạng thái	Sử dụng ROOM.status = 'available' để tự động hiển thị phòng trống. Có thể thêm bảng ROOM_POSTINGS để đăng tin, nhưng không bắt buộc nếu dùng status.

Đề xuất ERD hoàn chỉnh

Đề xuất bổ sung các bảng và khóa ngoại như sau:

- **Bảng TENANTS**: Lưu thông tin khách thuê. Ví dụ:

```
CREATE TABLE tenants (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(100) NOT NULL,
  cccd VARCHAR(20) UNIQUE,
  phone VARCHAR(20),
  deposit_amount NUMERIC(12,0) DEFAULT 0,
  checkin_date DATE,
  note TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ
);
```

- **Bảng BUILDINGS**: Tòa nhà hoặc khu trọ. Ví dụ:

```
CREATE TABLE buildings (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(100),
  address TEXT,
  electricity_price NUMERIC(10,2),
  water_price NUMERIC(10,2),
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ
);
```

- **Bảng ROOMS**: Phòng trọ.

```
CREATE TABLE rooms (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  building_id UUID REFERENCES buildings(id) ON DELETE CASCADE,
```

```

number VARCHAR(20) NOT NULL,
price NUMERIC(12,0) NOT NULL,
status VARCHAR(20) CHECK (status IN
('available','occupied','maintenance')),
internet_fee NUMERIC(12,0) DEFAULT 0,
description TEXT,
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ
);
CREATE INDEX ON rooms(status);

```

• **Bảng** `CONTRACTS` : Hợp đồng thuê.

```

CREATE TABLE contracts (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID REFERENCES tenants(id) ON DELETE SET NULL,
  room_id UUID REFERENCES rooms(id) ON DELETE SET NULL,
  start_date DATE NOT NULL,
  end_date DATE,
  deposit_amount NUMERIC(12,0) DEFAULT 0,
  status VARCHAR(20) CHECK (status IN
('active','expired','terminated')),
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ
);

```

• **Bảng** `INVOICES` : Hóa đơn.

```

CREATE TABLE invoices (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,
  invoice_date DATE NOT NULL,
  due_date DATE,
  total_amount NUMERIC(12,0) NOT NULL,
  status VARCHAR(10) CHECK (status IN ('pending','paid','overdue')),
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ
);
CREATE INDEX ON invoices(invoice_date);

```

• **Bảng** `INVOICE_ITEMS` : Chi tiết từng mục trên hóa đơn.

```

CREATE TABLE invoice_items (
  invoice_id UUID REFERENCES invoices(id) ON DELETE CASCADE,
  item_no SERIAL,
  description TEXT,
  amount NUMERIC(12,0) NOT NULL,
  vat_rate NUMERIC(3,2) DEFAULT 0,

```

```
PRIMARY KEY(invoice_id, item_no)
);
```

- **Bảng** `UTILITY_READINGS` : Ghi nhận chỉ số điện/nước.

```
CREATE TABLE utility_readings (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  room_id UUID REFERENCES rooms(id) ON DELETE CASCADE,
  meter_type VARCHAR(10) CHECK (meter_type IN ('electric', 'water')),
  prev_reading NUMERIC(10,2) NOT NULL,
  curr_reading NUMERIC(10,2) NOT NULL,
  reading_date DATE NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

- **Bảng** `SERVICES` : Dịch vụ kèm theo.

```
CREATE TABLE services (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(100) NOT NULL,
  price NUMERIC(12,0) NOT NULL,
  unit VARCHAR(50) DEFAULT 'month',
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ
);
```

- **Bảng** `TENANTS_VEHICLES` và `VEHICLES` :

```
CREATE TABLE vehicles (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  type VARCHAR(50),
  brand VARCHAR(50),
  color VARCHAR(30),
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE TABLE tenants_vehicles (
  tenant_id UUID REFERENCES tenants(id) ON DELETE CASCADE,
  vehicle_id UUID REFERENCES vehicles(id) ON DELETE CASCADE,
  license_plate VARCHAR(20) NOT NULL,
  parking_fee NUMERIC(10,0) DEFAULT 0,
  photo_url TEXT,
  PRIMARY KEY(tenant_id, vehicle_id)
);
```

- **Bảng** `PAYMENTS` : Giao dịch thanh toán.

```
CREATE TABLE payments (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```

invoice_id UUID REFERENCES invoices(id) ON DELETE CASCADE,
amount NUMERIC(12,0) NOT NULL,
method VARCHAR(20),
paid_at TIMESTAMPTZ NOT NULL,
reference TEXT,
created_at TIMESTAMPTZ DEFAULT now()
);

```

• **Bảng** `DEBTS` : Công nợ.

```

CREATE TABLE debts (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  invoice_id UUID REFERENCES invoices(id) ON DELETE CASCADE,
  amount_due NUMERIC(12,0) NOT NULL,
  due_date DATE,
  status VARCHAR(20) CHECK (status IN ('unpaid', 'partial', 'cleared')),
  created_at TIMESTAMPTZ DEFAULT now()
);

```

• **Bảng** `NOTIFICATIONS` : Nhắc nhở.

```

CREATE TABLE notifications (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID REFERENCES tenants(id),
  type VARCHAR(20),
  content TEXT,
  scheduled_time TIMESTAMPTZ,
  sent_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now()
);

```

• **Bảng** `AUDIT_LOG` : Lưu lịch sử thay đổi.

```

CREATE TABLE audit_log (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  table_name VARCHAR(50),
  record_id UUID,
  action VARCHAR(10), -- e.g., INSERT/UPDATE/DELETE
  old_data JSONB,
  new_data JSONB,
  action_time TIMESTAMPTZ DEFAULT now(),
  user_id UUID
);

```

• **Bảng** `ROOM_PHOTOS` : Ảnh phòng.

```

CREATE TABLE room_photos (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

```

```
room_id UUID REFERENCES rooms(id) ON DELETE CASCADE,
photo_url TEXT,
caption TEXT,
created_at TIMESTAMPTZ DEFAULT now()
);
```

Các bảng trên cho phép lưu trữ đầy đủ thông tin theo tính năng đã nêu (Dashboard, thống kê, thanh toán, nhắc nợ, v.v.). Chú ý việc khởi tạo cột **AUDIT** và **SOFT DELETE** (`deleted_at`) để theo dõi thay đổi và xóa mềm ⁴.

Hình 1. Ví dụ minh họa ERD cho hệ thống quản lý bất động sản (chỉ mang tính chất minh họa). Mô hình CSDL đề xuất sẽ bao gồm các bảng đã nêu ở trên với quan hệ 1-n giữa *Building-Rooms*, *Room-Contracts*, *Contract-Invoices*, *Invoice-InvoiceItems*, và sử dụng *tenant_id* để phân tách dữ liệu theo từng khách thuê (nếu có). Các chi tiết khác (như bảng bổ trợ, bảng RBAC) được mô tả trong DDL và phần dưới.

```
erDiagram
    TENANTS ||--o{ CONTRACTS : "ký hợp đồng"
    ROOMS ||--o{ CONTRACTS : "thuê bởi"
    CONTRACTS ||--o{ INVOICES : "phát sinh"
    INVOICES ||--o{ INVOICE_ITEMS : "bao gồm"
    TENANTS ||--o{ TENANTS_VEHICLES : "sở hữu"
    VEHICLES ||--o{ TENANTS_VEHICLES : ""
    CONTRACTS ||--o{ PAYMENTS : "thanh toán"
    INVOICES ||--o{ DEBTS : "nợ"
    ROOMS ||--o{ UTILITY_READINGS : "ghi chỉ số"
    ROOMS ||--o{ ROOM_PHOTOS : "có ảnh"
    USERS ||--o{ USER_ROLES : ""
    ROLES ||--o{ USER_ROLES : ""
    TENANTS ||--o{ NOTIFICATIONS : "nhận"
```

Logic tính tiền tự động và thanh toán QR

Quy trình lập hóa đơn diễn ra hàng tháng như sau: chủ trọ nhập chỉ số điện, nước mới cho từng phòng; phần mềm tính toán tự động:

- Tiền phòng:** giá thuê phòng (theo `contract`) đã cấu hình. Nếu khách vào giữa kỳ, tính *prorate* dựa trên số ngày ở thực tế (có thể tính theo tỷ lệ ngày so với tổng ngày tháng).
- Tiền điện:** `usage = cur_reading - prev_reading`. Ví dụ, nếu điện đầu tháng 200kWh, cuối tháng 300kWh và đơn giá 4.000đ/kWh, thì tiền điện = $(300-200)*4.000 = 400.000\text{đ}$.
- Tiền nước:** tương tự. Ví dụ, nước đầu 50m³, cuối 60m³, giá 10.000đ/m³, tiền = $(60-50)*10.000 = 100.000\text{đ}$.
- Các dịch vụ khác:** ví dụ Wifi cố định 100.000đ/tháng, vệ sinh 50.000đ, v.v., cộng trực tiếp. Có thể có VAT nếu áp dụng (cộng thêm 10%).
- Tổng hóa đơn** = tổng từng khoản trên. Kết quả làm tròn (ví dụ làm tròn đến 100đ để tiện tiền mặt).

Ví dụ cho 3 phòng:

- Phòng 101: Giá thuê 3.000.000đ; điện (200→250 kWh, giá 3.500đ) = 175.000đ; nước (100→120 m³, giá 10.000đ) = 200.000đ; Wifi = 100.000đ. *Hóa đơn tổng: 3.475.000đ (làm tròn 3.475.000đ).*
- Phòng 102: Giá thuê 2.500.000đ; điện (300→350, giá 3.500đ) = 175.000đ; nước (80→95, giá 10.000đ) = 150.000đ; Wifi = 100.000đ. *Hóa đơn tổng: 2.925.000đ.*
- Phòng 103: Giá thuê 3.200.000đ; điện (150→200, giá 3.500đ) = 175.000đ; nước (50→65, giá 10.000đ) = 150.000đ; Wifi = 100.000đ. *Hóa đơn tổng: 3.625.000đ.*

Hệ thống tạo hóa đơn có kèm **mã QR** thanh toán (Napas/ZaloPay/etc) với đúng tổng tiền. Khi khách quét QR và thanh toán, cổng thanh toán sẽ **POST callback** về hệ thống tại `callback_url` để thông báo trạng thái đã thanh toán ⁵. Ứng dụng nhận callback, so khớp `invoice_id` và `amount` (thường sử dụng tham số `app_trans_id` hoặc `reference` trả về) và tự động gạch nợ (cập nhật trạng thái hóa đơn thành “paid” và ghi vào bảng `PAYMENTS`). Nếu giao dịch chậm hay chưa khớp, phần mềm sẽ duy trì danh sách đối soát (ví dụ một job định kỳ so sánh sao kê ngân hàng với `pending invoices`). Quy trình này đảm bảo không cần thủ công đối chiếu.

Để nhắc khách thanh toán, có thể dùng **Zalo Notification Service (ZNS) API** – ví dụ gửi mẫu tin Zalo OA đến số điện thoại khách với nội dung nhắc hạn đóng tiền ⁶. Tính năng này tự động gửi khi hóa đơn được tạo (trên app/web).

Phân quyền và API chính

- **RBAC đa-tenant:** Sử dụng mô hình phân quyền theo tenant. Ví dụ theo WorkOS, bảng `roles` có thêm `tenant_id`, và bảng `user_roles(user_id, tenant_id, role_id)` gán quyền cho người dùng từng tenant ⁹ ¹². Mỗi truy vấn cần kiểm tra `tenant_id` (ví dụ trong RLS policy hoặc điều kiện WHERE). Các mẫu quyền (owner, admin, staff) có thể là mặc định hoặc cho phép tenant tự định nghĩa thêm (mô hình Hybrid RBAC). WorkOS khuyến nghị tạo *composite index* như `user_roles(user_id, tenant_id)` để tăng tốc tra quyền ².
- **Bảo mật dữ liệu (RLS):** Nếu ứng dụng phải hỗ trợ nhiều chủ trọ trên chung CSDL, dùng **Row-Level Security** của PostgreSQL để cô lập dữ liệu: thêm cột `tenant_id` vào tất cả bảng chính, và chính sách RLS `USING (tenant_id::text = current_setting('app.current_tenant'))` (hoặc so sánh với `current_user`) ³. Như AWS Database blog ghi nhận, RLS cho phép “chính sách bắt buộc cột tenant” như một **WHERE clause tự động** do DB điều khiển ³.
- **API RESTful chính:** Định nghĩa các endpoint CRUD cơ bản cho từng chức năng. Ví dụ:
- **Khách thuê/Tenants:** `GET/POST /api/tenants`, `GET/PUT /api/tenants/{id}`.
- **Phòng/Rooms:** `GET /api/rooms?status=available`, `POST /api/rooms`, `GET/PUT/DELETE /api/rooms/{id}`.
- **Hợp đồng/Contracts:** `POST /api/contracts`, `GET /api/contracts?tenantId=...`, `GET/PUT /api/contracts/{id}`.
- **Hóa đơn/Invoices:** `POST /api/invoices/generate` (tự tính và tạo cho kỳ này), `GET /api/invoices?month=...`, `GET /api/invoices/{id}`, `PUT /api/invoices/{id}/pay`.
- **Thanh toán/Payments:** `POST /api/payments` (xử lý callback), `GET /api/payments?invoiceId=...`.
- **Dịch vụ/Services:** `GET/POST /api/services`, `GET /api/services/{id}`.
- **Xe/Vehicle:** `GET/POST /api/vehicles`, `GET /api/tenants/{tenantId}/vehicles`.
- **Nhắc nhở/Notifications:** `POST /api/notifications/sendReminders`.

Endpoints sẽ làm việc trong bối cảnh `tenant_id` (để kiểm tra quyền/hạn mức). Mỗi endpoint cần validate dữ liệu, kiểm tra phân quyền (RBAC) và handle exceptions.

Chiến lược nhập dữ liệu (Excel) và di cư (Migration)

- **Xuất/nhập Excel:** Người dùng chỉ cần upload file Excel (CSV) theo mẫu (Room list, Tenant list, giá dịch vụ, nợ tồn, v.v.). Hệ thống chuyển đổi Excel thành CSV (ví dụ "Save As CSV" như hướng dẫn Atlassian ¹³) rồi dùng lệnh SQL `COPY` hoặc script ETL để nạp vào bảng tương ứng. Trước khi import, cần validate các cột (đảm bảo định dạng ngày, số, tham chiếu hợp lệ). Các bản ghi có thể dùng cơ chế "upsert" (`INSERT ... ON CONFLICT UPDATE`) để tránh trùng lặp nếu import nhiều lần (đảm bảo tính idempotency).
- **Khớp dữ liệu hiện tại:** Xác định bảng nguồn từ hệ thống cũ hoặc file cũ (như Danh sách phòng, Khách, Hợp đồng, Công nợ). Duyệt chuyển lần lượt: Nhà (BUILDINGS), Phòng (ROOMS), Dịch vụ (SERVICES), Khách thuê (TENANTS), Hợp đồng (CONTRACTS), Hóa đơn cũ (INVOICES + INVOICE_ITEMS), Thanh toán/ Công nợ (PAYMENTS/DEBTS). Kiểm tra mối quan hệ khoá ngoại trước khi nạp và tạo view/trigger tạm nếu cần map dữ liệu lẫn nhau. Lập danh sách kiểm tra (checklist) để chắc không sót bảng nào: ví dụ kiểm tra có bao nhiêu phòng, hợp đồng, v.v., rồi so sánh tổng trước và sau khi import để đảm bảo không mất bản ghi.

Chỉ mục và tối ưu truy vấn báo cáo

Cuối cùng, cần tạo chỉ mục trên các trường thường xuyên tìm kiếm/ JOIN để báo cáo nhanh. Ví dụ:

- **Các cột khoá ngoại:** tự động có chỉ mục khi tạo FK, nhưng nên tạo thêm chỉ mục riêng nếu cần (ví dụ `CREATE INDEX ON payments(invoice_id)`).
- **Trường lọc phổ biến:** index trên `rooms.status`, `invoices(invoice_date)`, `contracts(tenant_id, status)`, v.v., giúp lọc nhanh phòng trống, báo cáo doanh thu theo tháng ¹.
- **RBAC composite index:** như khuyến nghị, tạo `CREATE INDEX ON user_roles(user_id, tenant_id)` và `roles(tenant_id)` ².
- **Báo cáo tổng hợp:** Nếu cần tổng hợp lớn (như doanh thu theo tháng, lãi lỗ), cần nhắc sử dụng **VIEW** hoặc **MATERIALIZED VIEW** để tính trước (ví dụ tổng `SUM(amount)` theo kỳ), hoặc báo cáo ngoài DB với BI tool.

Việc thiết lập chỉ mục phù hợp là rất quan trọng để đảm bảo các truy vấn báo cáo (như Dashboard) trả kết quả trong vài giây, tránh truy vấn toàn bộ bảng lớn. Tóm lại, mô hình CSDL được đề xuất tuân thủ chuẩn hoá, có khoá chính/khoá ngoại rõ ràng và các ràng buộc hợp lý, giúp hệ thống quản lý trợ hoạt động ổn định, dễ bảo trì và mở rộng theo yêu cầu.

Nguồn tham khảo: Thiết kế này dựa theo các nguyên tắc chuẩn hoá dữ liệu và thực tiễn DBA (ví dụ: sử dụng khoá chính/phụ, ràng buộc CHECK, chỉ số) ¹ ⁷, mô hình RBAC đa-tenant phù hợp ⁹ ², cùng tài liệu PostgreSQL về RLS ³ và hướng dẫn API thanh toán (ZaloPay) ⁵, API nhắn tin Zalo OA (ZNS) ⁶. Các trích dẫn cụ thể được thể hiện ở trên.

¹ How to Design ER Diagrams for Real Estate Property Management - GeeksforGeeks
<https://www.geeksforgeeks.org/dbms/how-to-design-er-diagrams-for-real-estate-property-management/>

² ⁹ ¹⁰ ¹² How to design an RBAC model for multi-tenant SaaS — WorkOS
<https://workos.com/blog/how-to-design-multi-tenant-rbac-saas>

3 Multi-tenant data isolation with PostgreSQL Row Level Security | AWS Database Blog

<https://aws.amazon.com/blogs/database/multi-tenant-data-isolation-with-postgresql-row-level-security/>

4 8 11 Essential Database Design: Five Fields Every Table Must Have | Dip

<https://dipkumar.dev/posts/essential-db-design-1/>

5 Danh sách API - ZaloPay Developer

<https://developers.zalopay.vn/v2/general/overview.html>

6 Tổng quan về API Zalo và cách sử dụng hiệu quả - Your Digital Partner

<https://digibird.co/api-zalo-va-cach-su-dung-hieu-qua/>

7 How to Design an ER Diagram for an Invoice Management System

<https://www.red-gate.com/blog/erd-for-invoice-management/>

13 Importing Excel Data into MySQL | Atlassian

<https://www.atlassian.com/data/databases/excel-to-mysql>